

JavaScript & Node.js Essentials for Playwright

Cope Automation

Comments

Comments

1. What is a comment?
 - a. Comment provides a way to explain the code (e.g as a header for a function)
 - b. Comments are ignored by the interpreter
2. How many types of comments we have in JS?
 - a. Single line
 - b. Multi-line
 - c. Hashbank
3. What's its syntax?
 - a. `// <comment>`
 - b. `/ * <comment(s)> */`
 - c. `#!`

Variables

Variable Definition and Syntax

- I. What is a variable?
 - a. Variable is an **identifier/name** that **stores value/data**
2. What's its syntax?

```
{ var | let | const } {variableName} [= value]
```

- I. How do you read the above syntax?
 - a. Declare a variable and initialize it with a value

Variable Rules

1. Variable name should start with a letter, or underscore (), or dollar sign (\$)
2. System reserved keywords can't be used e.g. if, else, throw, break
3. Variable names are case-sensitive
4. Can't have same name for other types (e.g. function name, object name)
5. Multiple variables can be declared in single line (separated by comma)
6. When you just declare a variable, the data type will be undefined

var, let and const

#	Statement	Definition	Comments
1	var	Declares a variable, optionally initializing it to a value	This was the only var declaration statement before ES6. In ES6 (ECMAScript2015), let and const are introduced
2	let	Declares a block-scoped, local variable, optionally initializing it to a value	* Preferred wherever possible
3	const	Declares a block-scoped, read-only named constant.	

Rules:

1. **const** variable must be initialized with a value
2. **const** variable can not be re-assigned
3. Generally, **const** variables will be capitalized

Variable Scoping

- I. Why do you need to use with `let` or `const` over `var`?
 - a. Because `let` and `const` support block scoping

Data types

Data types

1. How many data types available in JavaScript?
 - a. A total of **8** types where **7** are primitives
2. What are the 8 data types?
 - a. string (e.g. "Hello")
 - b. number (e.g. 5, 10.5, 20)
 - c. boolean (true/false)
 - d. undefined
 - e. null
 - f. bigint
 - g. symbol (data type whose instances are unique and immutable)
 - h. Object type (Object, Array, Date..etc)

Data types literals and its notations

1. string
 - a. Single quote, double quotes, template literals (`````${}``)
2. number
 - a. Integer or floating points
3. boolean (true/false)
4. Object
 - a. `{}`
5. Array
 - a. `[]`
6. Regexp
 - a. `/`
7. Undefined and null

typeof operator and its use cases

1. What's use of typeof Operator?
 - a. To understand a data type of JS components
2. What's its syntax?
 - a. `typeof <any_valid_JS type>`
 - b. `typeof(any_valid_JS type)`

Different forms of data

1. Literals
2. Variable (Assigned to a variable)
3. Expression (Evaluates to, mostly using operators or return statement)

Truthy, falsy and nullish values

1. What are the falsy values?

- a. false
- b. undefined
- c. null
- d. 0
- e. NaN
- f. "" (empty string)

2. Boolean and non-boolean context of nullish value

Type	Boolean context (e.g. if condition)	Numeric context
undefined	false	NaN
null	false	0

Data types conversions

- I. The following inbuilt functions can be used to convert data types
 - a. `parseInt()`
 - b. `parseFloat()`
 - c. unary plus operator
 - d. `toString()` method

Data type - Summary points

1. A total of 8 data types where 7 primitives and 1 object type
2. `typeof` operator can be used to check a data type of variable/expression
3. The `typeof` operator returns a string value starting with lowercase
4. A data can be represented in three forms: *Literals, variables and expression*
5. Following 6 values will be considered falsy in a boolean context `false`, `undefined`, `null`, `0`, `NaN` and empty string
6. `parseInt()`, `parseFloat()`, `.toString()` method can be used to convert datatype
7. Be familiar with these notations: `{}`, `[]`, `/abc/`, `""`
8. All primitive data types are immutable
9. Why do you need to stick with `let` or `const`?
 - a. Because `let` and `const` support block scoping
10. Except for 'undefined' and 'null', every other 5 primitives have wrapper object types (starts with uppercase)

Operators

Operators - Intro and types

- I. Key things to note when using an operator
 - a. Operator: Understand the notation and syntax
 - b. Operands
 - c. Returned value

#	Operator type	Examples
1	Unary	typeof "Hello"
2	Binary	$x + y$
3	Ternary	isTrue ? Yes: No

How many ? & What are the most used ones?

#	Operator type	Operators	Examples
1	Assignment	=, +=, -=,	let count = 1
2	Arithmetic	+, -, *, **, /, %, ++, --	for (let i = 1; i <= 5, i++) { }
3	Comparison	==, ===, !=, !==, >, <, <=, >=	
4	Logical	&&, , !	
5	String	+	
6	Ternary	?	
7	Unary	typeof, delete, void	
8	Relational	in, instanceof	Returns a boolean
9	Comma		
10	Bitwise		

Strict mode

Strict mode

1. What's strict mode?
 - a. Strict mode prevents semantics(syntax related) error in JavaScript
2. What's the use case?
 - a. Prevents any undeclared global variable
 - b. Use of implements, interface, let, package, private, protected, public, static, and yield as identifiers
3. How to turn on strict mode?
 - a. `"use strict"`

Conditional Statements (If...else & switch case)

How do you read these?

[Example 1]

```
If (status = "PASS") {}
```

[Example 2]

```
If (status === "PASS") {}
```

Syntax

```
if (condition) {  
    statement_1;  
} else {  
    statement_2;  
}
```

Evaluates to true

```
if (condition_1) {  
    statement_1;  
} else if (condition_2) {  
    statement_2;  
} else if (condition_n) {  
    statement_n;  
} else {  
    statement_last;  
}
```

Falsy values

The following values evaluate to `false` (also known as [Falsy](#) values):

- `false`
- `undefined`
- `null`
- `0`
- `NaN`
- the empty string (`"`)

All other values—including all objects—evaluate to `true` when passed to a conditional statement.

Switch case [break, default]

1. What does switch case do?

- a. It tries to match given expression with case labels and executes the program

2. What's the syntax?

A `switch` statement looks like this:

```
switch (expression) {  
    case label_1:  
        statements_1  
        [break;]  
    case label_2:  
        statements_2  
        [break;]  
    ...  
    default:  
        statements_def  
        [break;]  
}
```

Switch case

- I. When to use switch case over if..else?
 - a. When the inputs/cases are known (useful for a predefined set of cases)
- I. What is the difference between if..else and switch case
 - a. If..else requires expression to evaluate to true/false but not for switch...case
- I. Rules:
 - a. Remember to include `break` statement at each case block
 - b. `default` statement will be executed if no previous cases are matched
 - c. `default` is optional and generally written at the end of all cases
 - d. Remember these four statements: `switch`, `case`, `break` and `default`

Loops

Loops Introduction

1. What are loops used for?
 - a. To iterate over a collection (more than one element)
2. What are the basic use cases?
 - a. To find a SPECIFIC element/member from a collection
 - b. Do something with ALL elements
3. General rules
 - a. Loop should end - check condition
 - b. If you initiate a variable inside a loop, it reset for every iteration

Different types of loops

[(Standard) for loop]

[forEach]

[while]

[do while]

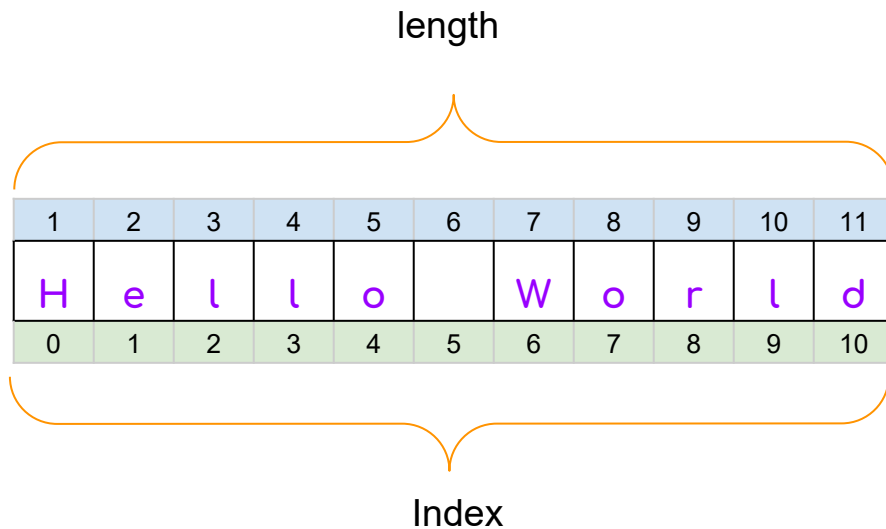
[for in]

[for of]

String Manipulation

String - Basics

- I. What is a string?
 - a. It is a sequence of characters (text value)
- I. What are the core characteristics of a string?



General rules:

1. Index starts from 0; each char can be accessed by knowing its index
2. `length` returns the total number of characters

Use of backslash

Use of backslash (\) in string

- I. What is backslash is called?
 - a. Escape character
2. What are the rules?
 - a. When the interpreter hits the backslash in string, it looks for the next character
 - i. If next char is a letter and has a special meaning(e.g. tab, new line), then it does the special thing
 - ii. If the next char is a letter and does not mean anything, the \ is skipped from the string
 - iii. If the next char is a special char(' , " , \), then the special char will be included in the string

What are those special chars and its meaning?

Character	Meaning
\b	Backspace
\f	Form feed
\n	New line
\r	Carriage return
\t	Tab
\v	Vertical tab
\'	Apostrophe or single quote
\"	Double quote
\\	Backslash character

Functions

Function - Introduction

I. What is a function?

- a. A set of statements to perform a task or calculate a value - similar to a procedure but function takes input and returns an output

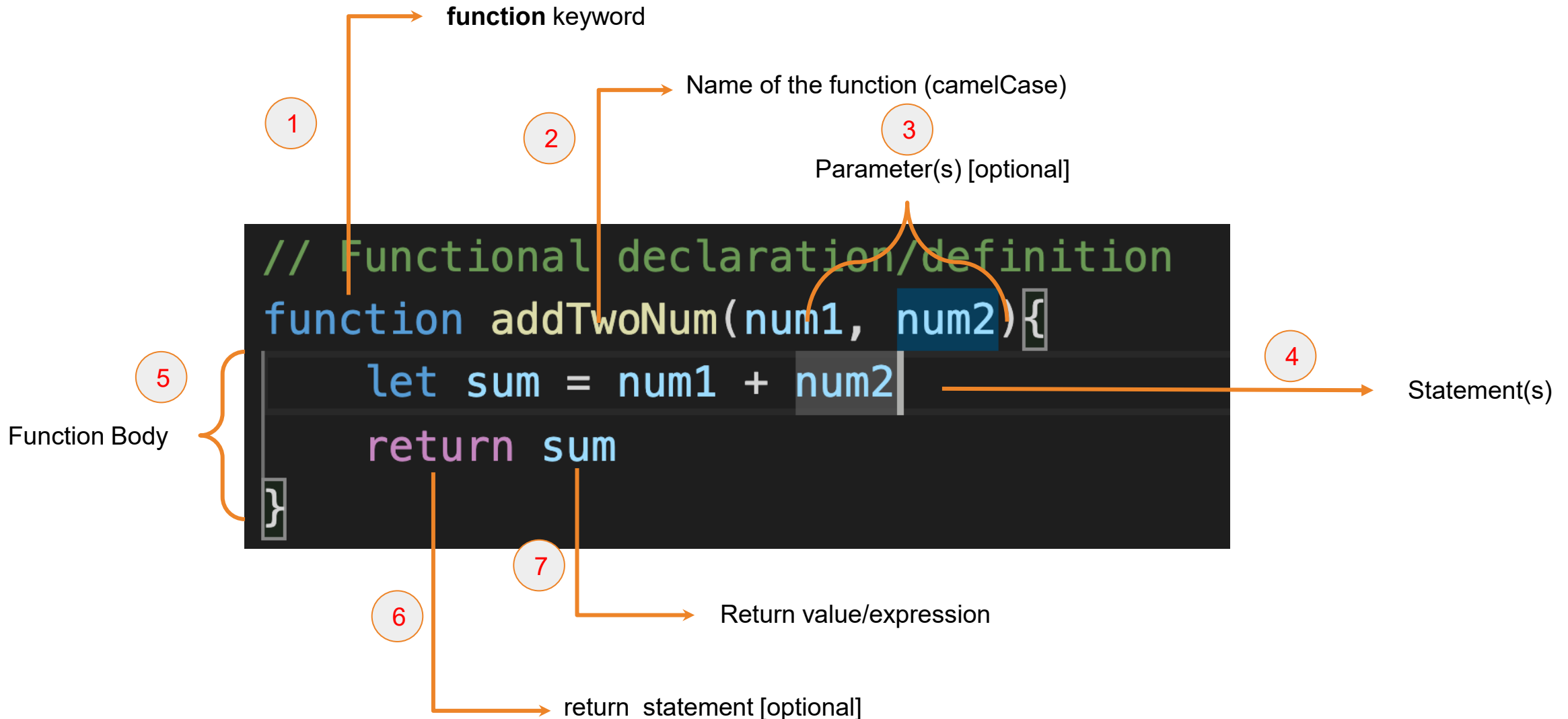
I. What are the two main stages of a function?

- a. Declare/define
- b. Calling

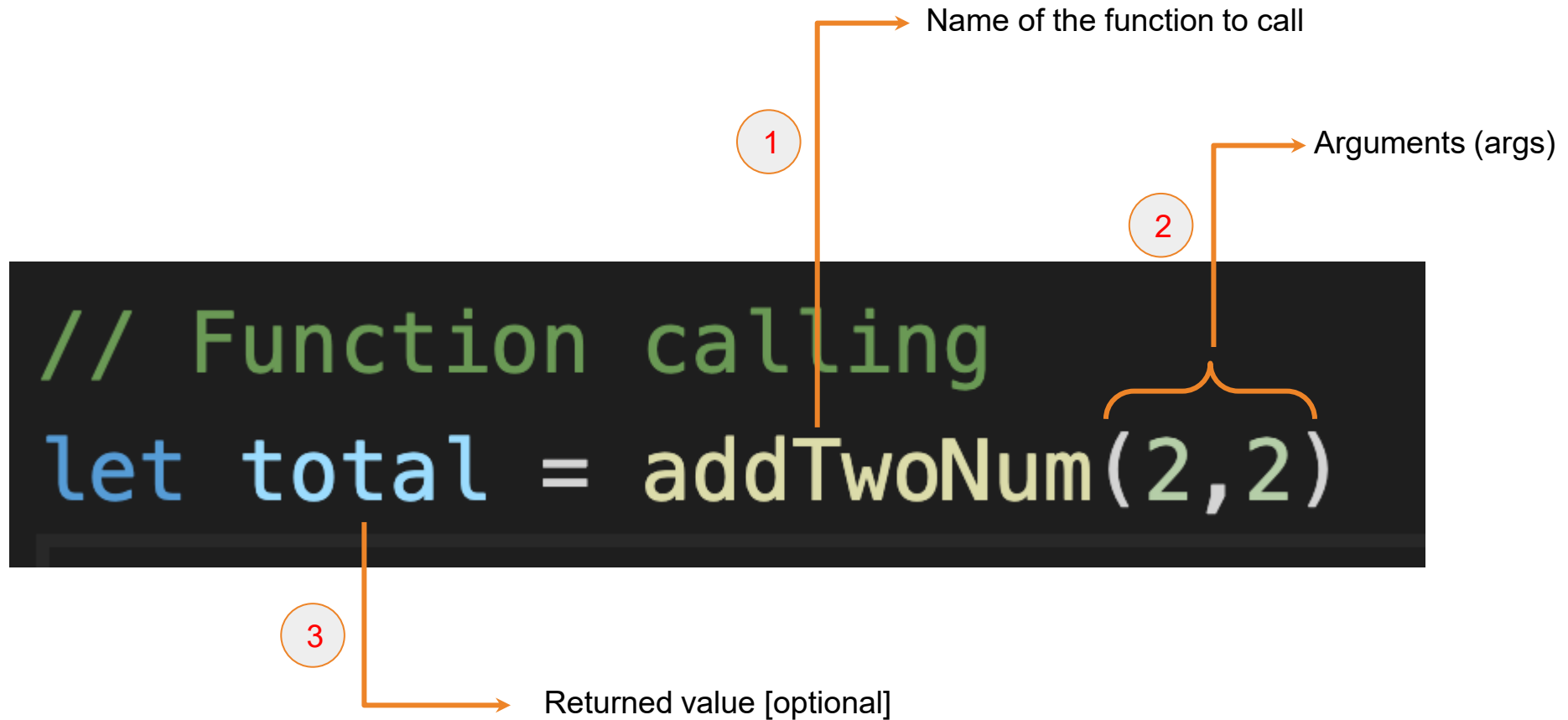
I. What are the two main types of functions?

- a. Named function (aka function declaration/definition)
- b. Anonymous function (aka function expression)
 - i. Callback function, arrow function

Function definition - Basic Syntax



Function calling - Basic Syntax



General notes on function...

I. Parameter Vs Argument?

- a. When you define a function the required values are called as parameter(s) (aka params) and when the function is called with actual values, then the actual values are called as argument(s) (aka args)

I. Remember these points

- a. You can provide any type of data as a function arg (no checking is done)
- b. You can provide more or less args when calling a function - no check is performed on # number of parameter(s) Vs Number of args received

[Fat] Arrow($\Rightarrow$) Functions

Arrow function

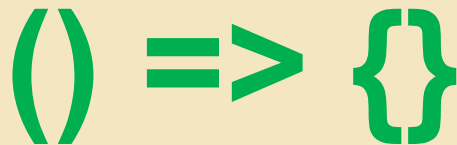
I. What's [fat] arrow function?

- a. It's a shorter version of function expression

I. What's the real time use case?

- a. Used mostly in callback context

I. What's its basic syntax?



`() => {}`

Arrow function rules

I. What are the rules?

- a. No “function” keyword
- b. `=>` is must and left and right of arrow is context-driven
- c. Arrow function can be called when it's assigned it to a variable
- d. No binding of *this* keyword
- e. Rules around syntax

#	Notations	Should present	Can skip
1	()	If zero or more param	If single param
2	<code>=></code>	Always present	
3	{ }	More than one statement	If single statement/expression
4	return	If more than one statement	If just single statement